Data ingestion architecture

4-7-25

Dani - ML Team

Table of Contents

What is the task?

Build the data reading compute part for a Data Ingestion Pipeline.

In other words:

Load training dataset stored in an S3 bucket to an EC2 machine ready for training.

Why?

Why?

- EBS storage is not cheap for constant storage.

Why?

- EBS storage is not cheap for constant storage.
- Standarization of our train pipelines to keep up with others

📝 ML Lifecycle

ML-Lifecycle-Detail.jpg

📝 Data preparation environment

Requirements:

Data preparation environment

Requirements:

- Dataset easy manipulation and modification (polars, pandas, ...) for:
  - Data aggregation
  - Data balancing
  - Data cleaning/normalization/regularization
  - Data labeling

Data preparation environment

Requirements:

- Dataset easy manipulation and modification (polars, pandas, ...) for:
    - Data aggregation
    - Data balancing
    - Data cleaning/normalization/regularization
    - Data labeling
- Dataset proper storage (DVC, AWS tools, polars,)

Data preparation environment

Requirements:

- Dataset easy manipulation and modification (polars, pandas, ...) for:
  - Data aggregation
  - Data balancing
  - Data cleaning/normalization/regularization
  - Data labeling
- Dataset proper storage (DVC, AWS tools, polars,)
- Pipeline creation (DVC, Dagger, Airflow,)

Data preparation environment

Requirements:

- Dataset easy manipulation and modification (polars, pandas, ...) for:
  - Data aggregation
  - Data balancing
  - Data cleaning/normalization/regularization
  - Data labeling
- Dataset proper storage (DVC, AWS tools, polars,)
- Pipeline creation (DVC, Dagger, Airflow,)
- Dataset versioning (DVC,)

📝 Training infra

Requirements:

📝 Training infra

Requirements:

- Experiment tracking (SageMaker, MLFlow)

📝 Training infra

Requirements:

- Experiment tracking (SageMaker, MLFlow)
- Dataset versioning (DVC,)

📝 Training infra

Requirements:

- Experiment tracking (SageMaker, MLFlow)
- Dataset versioning (DVC,)
- Data ingestion (AWS storage tools, polars, ...)

📝 Training infra

Requirements:

- Experiment tracking (SageMaker, MLFlow)
- Dataset versioning (DVC,)
- Data ingestion (AWS storage tools, polars, ...)
- Training environment (uv, pytorch, ...)

Considerations

 Training datasets processing best practices

E.g.: `train_21082019.txt`

# samples	size (GB)	mean sample size (KB)	Usual training batch size (samples)	Usual training batch size (MB)
1180347	38.5	~32.61	32	1.04

Best practices for high performance datasets processing

Best practices for high performance datasets processing

- Sequential I/O

Best practices for high performance datasets processing

- Sequential I/O
- Pipelining

Best practices for high performance datasets processing

- Sequential I/O
- Pipelining
- Sharding

Best practices for high performance datasets processing

- Sequential I/O
- Pipelining
- Sharding
- Parallelization

Best practices for high performance datasets processing

- Sequential I/O
- Pipelining
- Sharding
- Parallelization
- High-Throughput Data Formats

Best practices for high performance datasets processing

- Sequential I/O
- Pipelining
- Sharding
- Parallelization
- High-Throughput Data Formats
- Dedicated Data Processing Workers

Sequential I/O

Characteristics:

Sequential I/O

sequential-io-intuition.png

Characteristics:

- All samples are together in memory, as in one file

Sequential I/O

sequential-io-intuition.png

Characteristics:

- All samples are together in memory, as in one file
- Already ordered in disk. Randomization occurs before dataset file creation

Sequential I/O

Performance difference:

Sequential I/O

Performance difference:

- 10x faster on usual local storage

Sequential I/O

sequential-io-intuition.png

Performance difference:

- 10x faster on usual local storage
- Allow simpler/faster network storage

Pipelining

pipelining_example.png

Sharding

Split a huge dataset file in several sequential parts.

Main advantage: allow **parallelization**

The story

📖 Solutions considered

- Local - files dataset
- S3 - files dataset
- S3 - WebDatasets
- Local - polars LazyFrames from parquet highly compressed
- S3 - polars LazyFrames from parquet highly compressed

📖 Custom benchmark framework

Developed a benchmark framework

📖 Custom benchmark framework

Developed a benchmark framework

- File Loader

📖 Custom benchmark framework

Developed a benchmark framework

- File Loader
- Ubiquitous Dataset

📖 Custom benchmark framework

Developed a benchmark framework

- File Loader
- Ubiquitous Dataset
- Benchmark class

fileLoader Abstraction

```python
class fileLoader(ABC):
  @abstractmethod
  def load(
      self, file_path: Path, *args, **kwargs
  ) -> IO | IOBase | ContextManager | pl.LazyFrame:
      pass
```

fileLoader example for local files

```python
@dataclass
class FSFileLoader(fileLoader):
    def load(
        self, file_path: Path, mode: str = "r", encoding: Opti
    ) -> IO:
        logging.info(f"Loading {file_path}...")
        return file_path.open(
            mode=mode,
            encoding=encoding,
        )
```

fileLoader example for S3 parquet files

```python
@dataclass
class S3ParquetLoader(fileLoader):
    s3_bucket_name: str

    def load(
        self,
        file_path: Path,
        schema: Optional[Dict] = None,
        aws_region: str = "eu-central-1",
    ) -> pl.LazyFrame:
        logging.info(f"Loading {file_path}...")
        return pl.scan_parquet(
            f"s3://{self.s3_bucket_name}/{file_path}",
            schema=schema,
            storage_options={"aws_region": aws_region},
```

Ubiquitous Dataset Abstraction

Load a custom Pytorch Dataset no matter where/ how the samples are stored.

```python
from torch.utils.data import Dataset
import polars as pl

class UbiquitousDataset(Dataset):
    @abstractmethod
    def load_dataset(
        self, file_path: Path, *args, **kwargs
    ) -> List[List[str]] | pl.LazyFrame:
        pass

```

```
11      @abstractmethod
12      def length(self) -> int:
13          pass
14
15      @abstractmethod
```

Ubiquitous Dataset Abstraction

Load a custom Pytorch Dataset no matter where/ how the samples are stored.

- Just needs the length calculation and the iter functionality

```python
from torch.utils.data import Dataset
import polars as pl

class UbiquitousDataset(Dataset):
    @abstractmethod
    def load_dataset(
        self, file_path: Path, *args, **kwargs
    ) -> List[List[str]] | pl.LazyFrame:
        pass

```

```
11      @abstractmethod
12      def length(self) -> int:
13          pass
14
15      @abstractmethod
```

Ubiquitous Dataset Abstraction

Load a custom Pytorch Dataset no matter where/ how the samples are stored.

- Just needs the length calculation and the iter functionality
- Inherits pytorch Dataset class as parent so it's automatically compatible with pytorch

```python
from torch.utils.data import Dataset
import polars as pl

class UbiquitousDataset(Dataset):
    @abstractmethod
    def load_dataset(
        self, file_path: Path, *args, **kwargs
    ) -> List[List[str]] | pl.LazyFrame:
        pass

```

```
11      @abstractmethod
12      def length(self) -> int:
13          pass
14
15      @abstractmethod
```

Ubiquitous Dataset Abstraction

Load a custom Pytorch Dataset no matter where/ how the samples are stored.

- Just needs the length calculation and the iter functionality
- Inherits pytorch Dataset class as parent so it's automatically compatible with pytorch

```python
from torch.utils.data import Dataset
import polars as pl

class UbiquitousDataset(Dataset):
    @abstractmethod
    def load_dataset(
        self, file_path: Path, *args, **kwargs
    ) -> List[List[str]] | pl.LazyFrame:
        pass

```

```
11      @abstractmethod
12      def length(self) -> int:
13          pass
14
15      @abstractmethod
```

S3ParquetDataset example

```
class S3ParquetDataset(UbiquitousDataset):
  def __init__(
      self,
      s3_bucket_name: str,
      dataset_file_path: Path,
      _samples_dir_path: Optional[Path] = None,
  ) -> None:
      self.s3_bucket_name = s3_bucket_name
      self.file_loader: S3ParquetLoader = S3ParquetLoader(s3_buck
      self.dataset_file_path = dataset_file_path
      self._samples_dir_path = _samples_dir_path

      self.items: pl.DataFrame = self.load_parquet_dataset().coll
          engine="streaming"
      )
```

Benchmark class

- Mock training (just iterating through samples using the Dataloader default way)
- Timer

```
def train_mock(self) -> float:
      t = Timer(name=f"{self.load_from}-{self.load_as}-train-m
      sum_t = 0
      for epoch in range(self.train_epochs):
          t.start()
          # for train_features, train_labels in tqdm(self.data
          for train_features, train_labels in tqdm(self.datalo
              pass
          sum_t += t.stop()
      mean_t = sum_t / self.train_epochs
      return mean_t
```

Results - Benchmarks

- Not real Training
- All following benchmarks have been calculated from wiris laptop

📖 Local - files dataset

📖 Local - files dataset

- No sequential I/O

Results

Size (Cloud storage)	Dataset creation time	Size (Local Disk)	Download time b4 training	Training (1 epoch) time
~40 GB	0s (already created)	~40 GB	15325.533 s	61.3527 s

📖 S3 - files dataset

📖 S3 - files dataset

- No sequential I/O

Results

...

Size (Cloud storage)	Dataset creation time	Size (Local Disk)	Download time b4 training	Training epoch) time
~40 GB	0s (already created)	0 GB	0 s	22223.2 s

📖 S3 - WebDatasets

📖 S3 - WebDatasets

- Sequential I/O

📖 S3 - WebDatasets

- Sequential I/O
- Pipelining

📖 S3 - WebDatasets

- Sequential I/O
- Pipelining
- Sharding

📖 S3 - WebDatasets

- Sequential I/O
- Pipelining
- Sharding
- Allows streaming and caching

📖 S3 - WebDatasets

- Sequential I/O
- Pipelining
- Sharding
- Allows streaming and caching
- Old
  - Not supported by Pytorch anymore
  - Not direct support to s3 buckets (needs a public url)
  - Issues

Results

Size (Cloud storage)	Dataset creation time	Size (Local Disk)	Download time b4 training	Training (1 epoch) time
~40 GB*	?	0 GB	0.0010 s ?	? s

*Tar compression may be improved

📖 Local - polars LazyFrames from parquet highly compressed

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a high compression

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a high compression

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed
- Able to export to any type, even pil objects, numpy and tensors

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed
- Able to export to any type, even pil objects, numpy and tensors
- Optimized loading and exporting functions to cloud storage

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a high compression

📖 Local - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed
- Able to export to any type, even pil objects, numpy and tensors
- Optimized loading and exporting functions to cloud storage
- Can share workload with GPU

Note: I found some difficulties testing different parquet files cases as the generation and later transmission to S3 was not working if not using a high compression

Results

Size (Cloud storage)	Dataset creation time	Size (Local Disk)	Download time b4 training	Training (1 epoch) time
3 GB	22.6430 s	3 GB	309.793 s	277.5 s

😏

📖 S3 - polars LazyFrames from parquet highly compressed

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed
- Able to export to any type, even pil objects, numpy and tensors

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed
- Able to export to any type, even pil objects, numpy and tensors
- Optimized loading and exporting functions to cloud storage

📖 S3 - polars LazyFrames from parquet highly compressed

- Sequential I/O
- Pipelining and parallelization ($scan_{async}$)
- Sharding
- Avoid memory consummption that is not needed
- Able to export to any type, even pil objects, numpy and tensors
- Optimized loading and exporting functions to cloud storage
- Can share workload with GPU

Results

Size (Cloud storage)	Dataset creation time	Size (Local Disk)	Download time b4 training	Train (1 epoch time
3 GB	403.6961 s	0 GB	0 s	249.4 s
	368.1926 s (streaming)			

😏

Brainstorming